National University of Computer and Emerging Sciences
Islamabad Operating System Lab Spring 2025

Lab Task 11

Lab Task 01: Student Quiz Evaluator (Joinable Threads)

Objective:

Use joinable threads to evaluate students' quiz results and collect output in the main thread.

Scenario:

Each student thread calculates whether the student passed (score ≥ 5) and returns the result.

Instructions:

- Define struct `Student` with name and score.
- Input data for 4 students.
- Create one thread per student.
- Each thread:
    - Checks if score ≥ 5.
    - Returns `"Pass"` or `"Fail"` using `pthread_exit()`.
- Main thread uses `pthread_join()` to collect results and display them.

Lab Task 02: Auto-Saver Simulation (Detached Threads)

Objective:

Use detached threads to simulate a background auto-save feature for an editor.

Scenario:

Auto-saver saves three parts of a document — "Header", "Body", and "Footer" — independently, and main thread doesn't wait.

Instructions:

- Create 3 threads for "Header", "Body", and "Footer".
- Immediately detach each using `pthread_detach()`.
- Each thread:
    - Sleeps for 1–2 seconds (simulate delay).
    - Prints "Saved [section]".
- Main thread ends after creating the threads.

Lab Task 03: Countdown Timer with Cancel Option (Thread Cancellation)

Objective:

Practice thread cancellation by stopping a countdown thread before it completes.

Scenario:

A thread counts down from 10. The user decides to cancel it after a few seconds.

Instructions:

- Create a countdown thread that:
    - Loops from 10 to 1, printing the number every 1 second.

    - Checks for cancellation using `pthread_setcancelstate()` and `pthread_testcancel()`.

- After 3 seconds, main thread calls `pthread_cancel()` on it.
- Print message if the countdown was interrupted.

Lab Task 04: Weather Alert System (Detached Thread)

Objective:

Use thread attributes to create a detached thread that makes independent decisions based on weather temperature.

Scenario:

A weather alert system checks the temperature and gives a message based on these rules:

- If temp ≥ 40 → print `"Heatwave Alert"`
- If temp ≤ 5 → print `"Cold Wave Alert"`
- Else → print `"Weather is Normal"`

Instructions:

- Input a temperature value from the user in `main()`.
- Create a thread with **detached attributes** (`pthread_attr_t`) and pass the temperature to it.
- Inside the thread:
    - Use simple `if-else` statements to analyze the temperature and print the message.
    - Sleep for 2 seconds to simulate processing delay.

- Main thread should **not wait** for this thread and should finish after starting it.

THE END
